



בחינה בעיבוד תמונות, חלק 2, 18/02/24

ענו על שתי השאלות. חומר פתוח ללא מחשבים או מחשבוני.
הערה: תשובה לשאלה יכולה לכלול יותר מנושא בודד המכוסה בבחינה.
משך הבחינה: שעה.

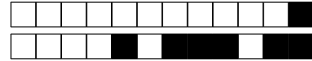
פתחו והסבירו את התשובה במחברת לפני הסימון בטופס הבחינה. סימון התשובות נעשה
ע"י X במשבצת המתאימה. ניתן לתקן סימון ע"י השחרת המשבצת וסימון X במשבצת
אחרת.

☐ 0	☐ 0	☐ 0	☐ 0	☐ 0	☐ 0	☐ 0	☐ 0	☐ 0
☐ 1	☐ 1	☐ 1	☐ 1	☐ 1	☐ 1	☐ 1	☐ 1	☐ 1
☐ 2	☐ 2	☐ 2	☐ 2	☐ 2	☐ 2	☐ 2	☐ 2	☐ 2
☐ 3	☐ 3	☐ 3	☐ 3	☐ 3	☐ 3	☐ 3	☐ 3	☐ 3
☐ 4	☐ 4	☐ 4	☐ 4	☐ 4	☐ 4	☐ 4	☐ 4	☐ 4
☐ 5	☐ 5	☐ 5	☐ 5	☐ 5	☐ 5	☐ 5	☐ 5	☐ 5
☐ 6	☐ 6	☐ 6	☐ 6	☐ 6	☐ 6	☐ 6	☐ 6	☐ 6
☐ 7	☐ 7	☐ 7	☐ 7	☐ 7	☐ 7	☐ 7	☐ 7	☐ 7
☐ 8	☐ 8	☐ 8	☐ 8	☐ 8	☐ 8	☐ 8	☐ 8	☐ 8
☐ 9	☐ 9	☐ 9	☐ 9	☐ 9	☐ 9	☐ 9	☐ 9	☐ 9

← Please encode your ID on the left by selecting **one digit per column**. The exam will not be graded if the ID is not encoded properly!

Please write your ID also in the box below.

Student ID:



Question 1

הסתכלו על הקוד להלן (שימו לב לטבלה למטה עם תיעוד לחלק מהפונקציות). יש לציין שהקוד כתוב כפי שלמדנו בכיתה, מלבד (1) ציון קרנל בניית פרמידות (בשורה 3) וקרנל שחזור פרמידות (שניתן ב-main) ו-(2) תוספת של coeff בשורה 14 לבניית התמונה -

1. **def main(reconstruction_kernel, coeff, levels):**
2. `lm1, lm2, mask = read_images()`
3. `kernel =  $\frac{1}{16} \cdot (1, 4, 6, 4, 1)$`
4. `lap1 = build_laplacian_pyramid(lm1, levels, kernel)`
5. `lap2 = build_laplacian_pyramid(lm2, levels, kernel)`
6. `gaus_mask = build_gaussian_pyramid(mask, levels, kernel)`
7. `lap_out = [None] * levels`
8. **for** `i` **in** `range(levels)`:
 - a. `lap_out[i] = gaus_mask[i] * lap1[i] + (1-gaus_mask[i]) * lap2`
9. `blended = image_reconstruction(lap_out, reconstruction_kernel, coeff)`
10. `blended = (blended - min(blended))/(max(blended) - min(blended))` *## stretch between 0 and 1*
11. **return** `blended`

12. **def image_reconstruction(lap, reconstruction_kernel, coeff):**
13. `img = lap[-1] * coeff[-1]`
14. **for** `i` **in** `reversed(range(0, len(lap) - 1))`: *## [len(lap) - 2, len(lap) - 3, len(lap) - 3, ..., 0]*
 - a. `img = im_expand(img, reconstruction_kernel) + (lap[i] * coeff[i])`
15. **return** `img`

כאשר:

חתימה	דוגמה למימדים	אלגוריתם
<code>Build_laplacian_pyramid(im, levels, kernel)</code>	Input: <code>im.shape=[N,N]</code> , <code>levels=3</code> Output: <code>[[N,N], [N/2,N/2], [N/4,N/4]]</code>	בניית פרמידת לפלסיאן
<code>Build_gaussian_pyramid(im, levels, kernel)</code>	Input: <code>im.shape=[N,N]</code> , <code>levels=3</code> Output: <code>[[N,N], [N/2,N/2], [N/4,N/4]]</code>	בניית פרמידת גאוסיאן
<code>im_expand(im, kernel)</code>	Input: <code>im.shape=[N,N]</code> Output: <code>[N*2,N*2]</code>	הגדלת תמונה פי 2

הנחות:

1. התמונות המקוריות והמסכה ריבועיות בגודל שהינו חזקה של 2.
2. הרמה הגבוהה ביותר בפירמידה לא קטנה מגודל מ-16X16.



+1/3/58+

סעיף א:

נרצה להריץ -

levels = 5

coeff = [1]*levels

main(reconstruction_kernel, coeff, levels)

לשם מילוי reconstruction_kernel, יש לבצע שתי בחירות - (1) קרנל ו-(2) מקדם לקרנל, כך שהפעלתם תוביל לתוצאה מיטבית.

לדוגמה, כדי להשתמש ב-reconstruction_kernel $\frac{1}{d} \cdot (a, b, c)$ נבחר בסעיף הראשון (a, b, c) ובסעיף השני נבחר $\frac{1}{d}$.

סעיף א.1:

בחרו קרנל ל-reconstruction_kernel:

☐ (1,1,1,1,1)

☐ (1,2,1)

☐ (1,-2,1)

☐ (1,-4,6,-4,1)

☐ (1,4,6,4,1)

סעיף א.2:

בחרו מקדם לקרנל (אותו נכפול ב-reconstruction_kernel לפני הפעלתו):

☐ 1/16

☐ 1/8

☐ 1/4

☐ 1/2



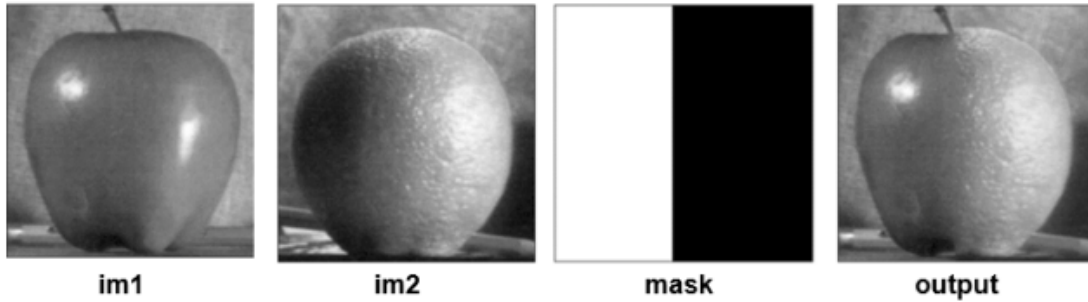
+1/4/57+

סעיף ב:

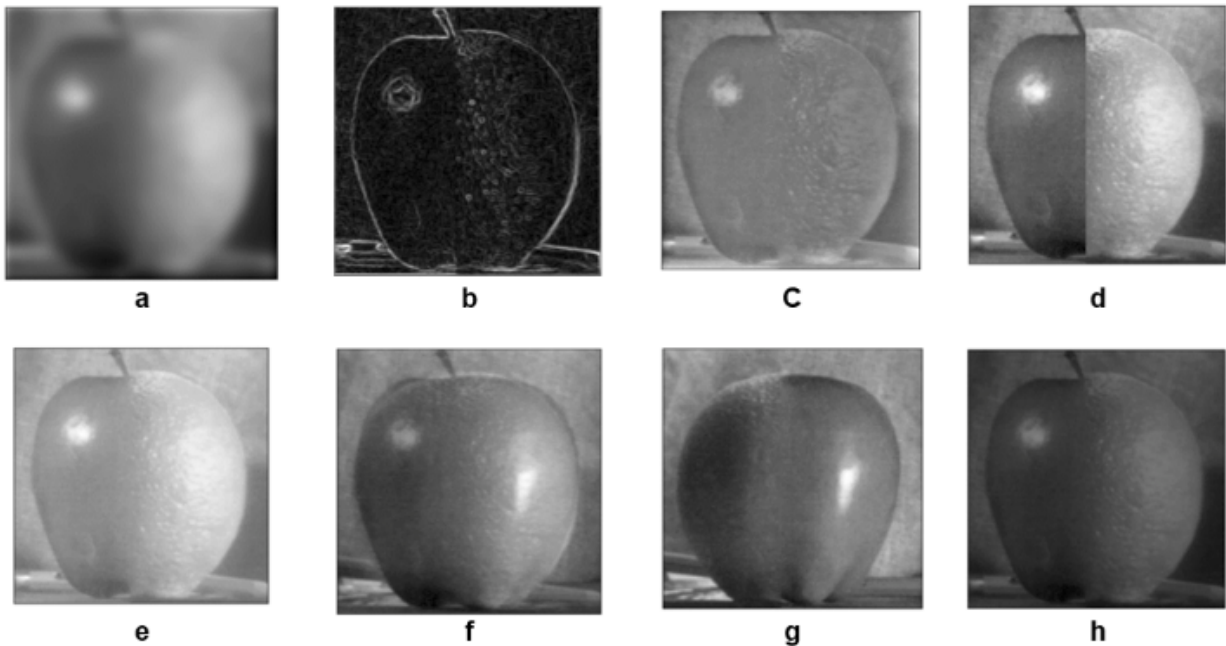
כאשר נריץ את הקוד הבא -

```
levels = 5
reconstruction_kernel = קרנל מיטבי
coeff = [1]*levels
main(reconstruction_kernel, coeff, levels)
```

עם התמונות הבאות בגודל 256×256 , נקבל את הפלט הבא -



בשני הסעיפים הבאים, נריץ בכל פעם קטע קוד. עליכם לבחור איזו מבין תמונות הפלט הבאות מתאימה לכל אחד מקטעי הקוד.



סעיף ב.1:

איזו תמונה, מבין האפשרויות לעיל, נקבל כאשר נריץ את הקוד הבא?

```
levels = 5
reconstruction_kernel = (לפי סעיף א) קרנל מיטבי
coeff = [0]*2+ [1]*(levels-2) ## ([0,0,1,1,1])
main(reconstruction_kernel, coeff, levels)
```

☐ a ☐ b ☐ c ☐ d ☐ e ☐ f ☐ g ☐ h



+1/5/56+

סעיף ב.2:

איזו תמונה, מבין האפשרויות לעיל, נקבל כאשר נריץ את הקוד הבא?

levels = 5

reconstruction_kernel = (לפי סעיף א)

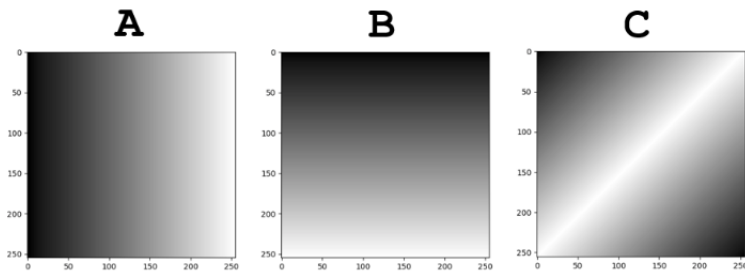
coeff = [1]*(levels-1) + [0] ## ([1,1,1,1,0])

main(reconstruction_kernel, coeff, levels)

☐ a ☐ b ☐ c ☐ d ☐ e ☐ f ☐ g ☐ h

סעיף ג:

להלן שלוש תמונות בגודל 256*256: שמוגדרות ע"י - $A[x,y]=x$; $B[x,y]=y$; $C[x,y]=\min\{x+y, 510-(x+y)\}$



התמונות מסודרות בזכרון שורה אחרי שורה -

0	1	...	255
256	257	...	511
...			
...			65025

סעיף ג.1:

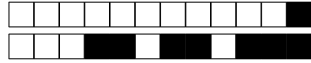
עבור אילו מהתמונות הנ"ל Huffman נותן דחיסה טובה יותר?

☐ Image A ☐ Image B ☐ Image C ☐ No difference

סעיף ג.2:

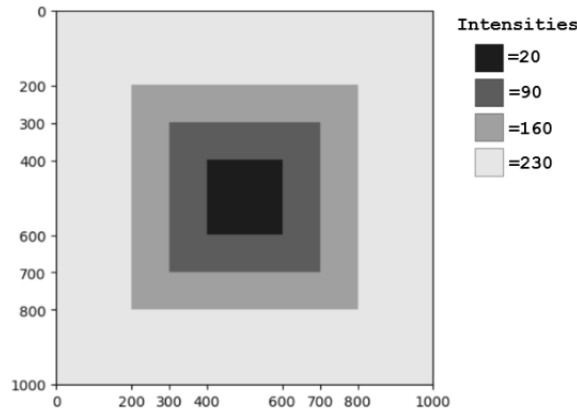
עבור אילו מהתמונות הנ"ל LZW נותן דחיסה טובה יותר?

☐ Image A ☐ Image B ☐ Image C ☐ No difference



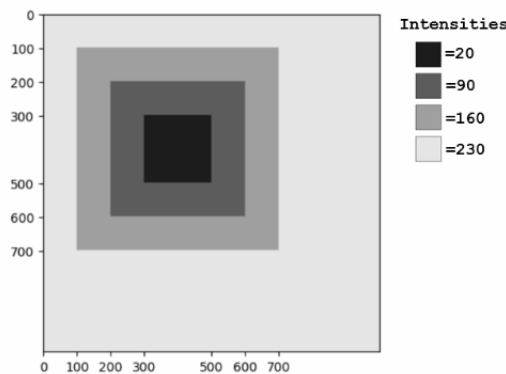
Question 2

נתונה לנו התמונה A הבאה בגודל 1000,1000 -



מתווסף לכל פיקסל ב A רעש אחיד ובלתי תלוי בטווח של ± 15 .

בנוסף, נתונה לנו תמונה B באותו הגודל, השונה ע"י ההזזה ציקלית בלבד לעומת התמונה הראשונה, כאשר בכל ציר ההזזה היא מספר ממשי שערכו המוחלט קטן מ-100. לדוגמה -



גם ל B נוסף רעש רנדומלי אחיד ובלתי תלוי בטווח של ± 15 (הרעש ב-A בלתי תלוי מהרעש ב-B).

עתה, נבצע את השלבים הבאים, כאשר עליכם לבחור בהמשך כיצד להשלים את המקומות המודגשים על מנת לשחזר באופן היעיל והמדויק ביותר את ההזזה:

1. נחשב לכל פיקסל בכל תמונה Harris (לא Scale invariant Harris) בסביבה של 13×13 פיקסלים.
2. נבחר מכל אחת מהתמונות את X הפיקסלים עבורם ציון ה-Harris הכי גבוה.
3. לכל a_i הפיקסלים שנבחרו בתמונה A נחשב נקודה תואמת b_i בתמונה B. התאמה (a_i, b_i) תסמן ההזזה $m_i = b_i - a_i$ (הערה: a_i, b_i, m_i כולם וקטורים דו ממדיים)
4. נבחר אלגוריתם לחשב מכל ההתאמות את ההזזה הנבחרת בין התמונות, נסמן את ההזזה שנבחרה ב- m^* .

סעיף א:

כמה פיקסלים תבחרו לשלב 2?

☐ 100

☐ 10

☐ 1

**סעיף ב:**

איזו התאמה תבחרו לשלב 3?

(1) לכל פיקסל a_i שנבחר עם Harris בתמונה A, נסתכל על כל הסביבות בגודל 13×13 בתמונה השנייה ונבחרסביבה $b_i = \operatorname{argmin}(\sum (w_A - w_B)^2)$, כאשר w_A מסמן את החלון בגודל 13×13 סביב a_i , ו- w_B מסמן את סביבה 13×13 בתמונה B שמרכזת b_i .(2) לכל פיקסל a_i שנבחר עם Harris בתמונה A, נסתכל על כל הסביבות בגודל 13×13 בתמונה השנייה, ונבחר

$$b_i = \operatorname{argmax}(NCC(w_A, w_B)) = \frac{\sum (w_A - \mu_{w_A})(w_B - \mu_{w_B})}{\sqrt{\sum (w_A - \mu_{w_A})^2 \sum (w_B - \mu_{w_B})^2}}$$

סביב a_i , ו- w_B מסמן את סביבה 13×13 בתמונה B שמרכזת b_i , ו- μ_{w_A}, μ_{w_B} הן התוחלות של חלונות W_A, W_B בהתאמה.(3) נחשב MOPS descriptor לכל X הנקודות a_i ולכל X הנקודות b_i . ההתאמה לכל פיקסל- a_i תהיה נקודת ה-שה- b_j descriptor שלה קרוב ביותר במרחק אוקלידי לדסקריפטור של a_i .(4) נחשב SIFT descriptor לכל X הנקודות a_i ולכל X הנקודות b_i . ההתאמה לכל פיקסל- a_i תהיה נקודת ה- b_j שה-descriptor שלה קרוב ביותר במרחק אוקלידי לדסקריפטור של a_i .☐ 1☐ 2☐ 3☐ 4**סעיף ג:**

איזה חישוב הזה נבחר לשלב 4?

(1) נדגום התאמה אחת רנדומלית, ונחשב ממנה את ההזזה.

(2) (i) נחזור איטרטיבית על צעד (ii) מספר פעמים השווה לחצי ממספר ההתאמות:

(ii) נדגום רנדומלית התאמה אחת שההזזה שלה היא m_k , ונחשב עבורה את מספר ההתאמות האחרותשההזזה שלהן מסכימה איתה: כאשר עבור הזזה m_k הסכמה מוגדרת בתור מספר ההתאמות (a_i, b_i) המקיימות $|(b_i - m_k) - a_i| \leq 2$.

(iii) עם סיום האיטרציות נבחר את ההזזה לה יש הכי הרבה הסכמות.

(3) נחשב את ההזזה להיות ממוצע ההזזות של כל ההתאמות, כלומר $\frac{1}{X} \sum_{i=1}^X m_i$.☐ 1☐ 2☐ 3☐ 4



+1/8/53+